

LabSoldier 作業指南書 — tsutsumi (在室判定バックエンド担当)

Hackathon Soldier Team

2026-05-21

目次

はじめに	2
第 1 部 共通の基礎知識	2
1.1 アプリの全体像	2
1.2 JavaScript / TypeScript の最低限	2
1.3 Node.js の基本	3
1.4 Express の基本	3
第 2 部 担当範囲と作成物	4
2.1 担当範囲	4
2.2 完成イメージ	5
第 3 部 SQLite と better-sqlite3	5
3.1 SQLite とは	5
3.2 better-sqlite3 ライブラリ	5
3.3 SQL の最低限	5
3.4 better-sqlite3 の使い方	6
第 4 部 実装手順	7
ステップ 0: ブランチを切る	7
ステップ 1: naganawa と user テーブルをすり合わせる	7
ステップ 2: ライブラリを入れる	8
ステップ 3: .env ファイルを作る	8
ステップ 4: DB セットアップ	8
ステップ 5: store.ts を SQLite ベースに置き換え	9
ステップ 6: IP 判定の改善	11
ステップ 7: タイムアウト処理	11
ステップ 8: presence ルートの認証	12
ステップ 9: 動作確認	12

ステップ 10: PR を出す	13
第 5 部よくある詰まりどころ	13
better-sqlite3 のインストールでエラー	13
SQLite ファイルが見つからない	13
IP が常に ::1 になる	13
CORS エラー	14
ping を打っても DB に書き込まれない	14
第 6 部ここまでやったら他の人と合流	14
付録 A: チートシート	14
付録 B: 用語集	15

はじめに

この PDF は tsutsumi が 在室判定バックエンドを実装するための指南書です。DB の選定から API の本実装、IP 判定、タイムアウト処理まで一通り解説しています。

第 1 部共通の基礎知識

1.1 アプリの全体像

[ブラウザ React] ⇄ [Node.js Express] ⇄ [DB (SQLite)]

frontend/ backend/ ← あなたの主戦場

ローカル: backend は http://localhost:3001、frontend は http://localhost:5173

1.2 JavaScript / TypeScript の最低限

変数

```
const name = "tsutsumi";    // 再代入できない
let count = 0;              // 再代入できる
```

関数とアロー関数

```
function add(a: number, b: number): number {
  return a + b;
}
const add = (a: number, b: number): number => a + b;
```

非同期処理 (DB 操作で多用する)

```
async function getUsers(): Promise<User[]> {  
  const result = await db.query("SELECT * FROM users");  
  return result.rows;  
}
```

// 呼ぶ側

```
const users = await getUsers();
```

配列メソッド

```
const nums = [1, 2, 3, 4, 5];  
nums.map(n => n * 2);           // [2, 4, 6, 8, 10]  
nums.filter(n => n > 2);        // [3, 4, 5]  
nums.find(n => n === 3);        // 3  
nums.reduce((a, b) => a + b, 0); // 15
```

オブジェクト

```
const user = { id: "u1", name: "tsutsumi" };  
const { id, name } = user;           // 分割代入  
const updated = { ...user, name: "x" }; // スプレッド
```

1.3 Node.js の基本

- ・ node script.js で実行
- ・ import でモジュール読み込み (今回 ESM)
- ・ process.env.XXX で環境変数読み取り

```
import express from "express";  
import dotenv from "dotenv";  
dotenv.config();  
  
const port = process.env.PORT ?? 3001;
```

1.4 Express の基本

Express は Node.js で HTTP サーバーを書くためのフレームワーク。

```
import express from "express";  
const app = express();  
app.use(express.json());
```

```

app.get("/hello", (req, res) => {
  res.json({ message: "hi" });
});

app.post("/users", (req, res) => {
  const { name } = req.body;
  // ...
  res.status(201).json({ id: "u1", name });
});

app.listen(3001, () => console.log("listening"));

```

Router を使う

```

import { Router } from "express";
export const presenceRouter = Router();

presenceRouter.get("/", (req, res) => { ... });
presenceRouter.post("/ping", (req, res) => { ... });

// index.ts で
app.use("/api/presence", presenceRouter);

```

Middleware

「リクエストを受けたあと、ルートハンドラを呼ぶ前に何かする関数」

```

app.use((req, res, next) => {
  console.log(req.method, req.url);
  next(); // 次のミドルウェアへ
});

```

第2部 担当範囲と作成物

2.1 担当範囲

ファイル	やること
backend/src/db/store.ts	インメモリ → SQLite に置き換え
backend/src/db/schema.sql (新規)	テーブル定義
backend/src/middleware/ipCheck.ts	LAB_ALLOWED_IPS を .env から読み込み
backend/src/routes/presence.ts	DB を使うように本実装
backend/src/lib/timeout.ts (新規)	10分 ping がなければ自動退室

2.2 完成イメージ

1. SQLite ファイル (backend/data/labsoldier.db) が自動生成される
2. /api/presence/ping を叩くと、IP ベースで在室判定して DB に書き込む
3. /api/presence で全員の在室状態が返る
4. /api/presence/manual で手動切替できる
5. 10 分 ping がないユーザーは自動で退室扱いになる

第 3 部 SQLite と better-sqlite3

3.1 SQLite とは

- ・ 1 ファイルで動くデータベース。サーバー不要、軽い
- ・ .db ファイルにテーブルとデータが入る
- ・ 本格的なアプリでは PostgreSQL に置き換えるが、ハッカソンには十分

3.2 better-sqlite3 ライブラリ

Node.js から SQLite を扱う一番速いライブラリ。同期的な API で読みやすい。

```
cd backend
npm install better-sqlite3
npm install -D @types/better-sqlite3
```

3.3 SQL の最低限

-- テーブル作成

```
CREATE TABLE users (
  id TEXT PRIMARY KEY,
  name TEXT UNIQUE NOT NULL,
  password_hash TEXT NOT NULL,
  avatar_id TEXT NOT NULL,
  created_at TEXT NOT NULL
);
```

-- 挿入

```

INSERT INTO users (id, name, password_hash, avatar_id, created_at)
VALUES ('u-tsutsumi', 'tsutsumi', 'hash', 'soldier-red', '2026-05-21T00:00:00Z');

-- 検索
SELECT * FROM users WHERE name = 'tsutsumi';
SELECT id, name FROM users ORDER BY created_at DESC LIMIT 10;

-- 更新
UPDATE presence SET is_present = 1, last_seen_at = '...' WHERE user_id = 'u-tsutsumi';

-- 削除
DELETE FROM users WHERE id = 'u-x';

```

3.4 better-sqlite3 の使い方

```

import Database from "better-sqlite3";
const db = new Database("data/labsoldier.db");

// テーブル作成 (起動時に一度)
db.exec(`
  CREATE TABLE IF NOT EXISTS users (
    id TEXT PRIMARY KEY,
    name TEXT UNIQUE NOT NULL,
    password_hash TEXT NOT NULL,
    avatar_id TEXT NOT NULL,
    created_at TEXT NOT NULL
  );
`);

// 挿入 (prepared statement)
const insert = db.prepare(`
  INSERT INTO users (id, name, password_hash, avatar_id, created_at)
  VALUES (?, ?, ?, ?, ?)
`);
insert.run("u-tsutsumi", "tsutsumi", "hash", "soldier-red", new Date().toISOString());

// 1 件取得
const findByName = db.prepare("SELECT * FROM users WHERE name = ?");
const user = findByName.get("tsutsumi"); // 該当なければ undefined

// 複数取得
const listAll = db.prepare("SELECT * FROM users");

```

```
const all = listAll.all();

// 更新
const updatePresence = db.prepare(`
  UPDATE presence SET is_present = ?, last_seen_at = ? WHERE user_id = ?
`);
updatePresence.run(1, new Date().toISOString(), "u-tsutsumi");
```

第 4 部 実装手順

ステップ 0: ブランチを切る

```
git switch main
git pull origin main
git switch -c feature/tsutsumi-presence-db
```

ステップ 1: naganawa と user テーブルをすり合わせる

naganawa が認証担当なので、users テーブルの設計を合わせる。

users テーブル（合意案）：

```
CREATE TABLE IF NOT EXISTS users (
  id TEXT PRIMARY KEY,
  name TEXT UNIQUE NOT NULL,
  password_hash TEXT NOT NULL,
  avatar_id TEXT NOT NULL,
  created_at TEXT NOT NULL
);
```

presence テーブル:

```
CREATE TABLE IF NOT EXISTS presence (
  user_id TEXT PRIMARY KEY,
  is_present INTEGER NOT NULL DEFAULT 0,    -- 0=false, 1=true
  source TEXT NOT NULL DEFAULT 'wifi',     -- 'wifi' | 'manual'
  entered_at TEXT,
  last_seen_at TEXT,
  FOREIGN KEY (user_id) REFERENCES users(id)
);
```

ステップ 2: ライブラリを入れる

```
cd backend
npm install better-sqlite3 dotenv
npm install -D @types/better-sqlite3
```

ステップ 3: .env ファイルを作る

backend/.env (gitignore 済み) :

```
PORT=3001
LAB_ALLOWED_IPS=127.0.0.1,::1
JWT_SECRET=dev-secret-change-me
```

backend/.env.example を更新するのも忘れずに。

ステップ 4: DB セットアップ

backend/src/db/database.ts (新規) :

```
import Database from "better-sqlite3";
import { mkdirSync } from "node:fs";
import { dirname } from "node:path";

const DB_PATH = "data/labsoldier.db";
mkdirSync(dirname(DB_PATH), { recursive: true });

export const db = new Database(DB_PATH);
db.pragma("journal_mode = WAL");
db.pragma("foreign_keys = ON");

// 起動時に一度走る
db.exec(`
  CREATE TABLE IF NOT EXISTS users (
    id TEXT PRIMARY KEY,
    name TEXT UNIQUE NOT NULL,
    password_hash TEXT NOT NULL,
    avatar_id TEXT NOT NULL,
    created_at TEXT NOT NULL
  );

  CREATE TABLE IF NOT EXISTS presence (
```



```

    user_id TEXT PRIMARY KEY,
    is_present INTEGER NOT NULL DEFAULT 0,
    source TEXT NOT NULL DEFAULT 'wifi',
    entered_at TEXT,
    last_seen_at TEXT,
    FOREIGN KEY (user_id) REFERENCES users(id)
  );
`);

```

```
console.log("[db] ready:", DB_PATH);
```

backend/.gitignore (あれば) に data/ を追加。なければ root の .gitignore に。

ステップ 5: store.ts を SQLite ベースに置き換え

backend/src/db/store.ts を全面書き換え:

```

import { db } from "../database.js";
import type { Presence, User } from "../types.js";

const userByIdStmt = db.prepare("SELECT * FROM users WHERE id = ?");
const allUsersStmt = db.prepare("SELECT * FROM users");
const presenceByIdStmt = db.prepare("SELECT * FROM presence WHERE user_id = ?");
const allPresencesStmt = db.prepare("SELECT * FROM presence");
const upsertPresenceStmt = db.prepare(`
  INSERT INTO presence (user_id, is_present, source, entered_at, last_seen_at)
  VALUES (?, ?, ?, ?, ?)
  ON CONFLICT(user_id) DO UPDATE SET
    is_present = excluded.is_present,
    source = excluded.source,
    entered_at = excluded.entered_at,
    last_seen_at = excluded.last_seen_at
`);

type UserRow = {
  id: string;
  name: string;
  avatar_id: string;
  password_hash: string;
  created_at: string;
};

type PresenceRow = {

```

```

    user_id: string;
    is_present: number;
    source: string;
    entered_at: string | null;
    last_seen_at: string | null;
};

function rowToUser(r: UserRow): User {
    return { id: r.id, name: r.name, avatarId: r.avatar_id };
}
function rowToPresence(r: PresenceRow): Presence {
    return {
        userId: r.user_id,
        isPresent: r.is_present === 1,
        source: r.source as "wifi" | "manual",
        enteredAt: r.entered_at,
        lastSeenAt: r.last_seen_at,
    };
}

export const store = {
    listUsers(): User[] {
        return (allUsersStmt.all() as UserRow[]).map(rowToUser);
    },
    getUser(id: string): User | undefined {
        const row = userByIdStmt.get(id) as UserRow | undefined;
        return row ? rowToUser(row) : undefined;
    },
    listPresences(): Presence[] {
        return (allPresencesStmt.all() as PresenceRow[]).map(rowToPresence);
    },
    getPresence(userId: string): Presence | undefined {
        const row = presenceByIdStmt.get(userId) as PresenceRow | undefined;
        return row ? rowToPresence(row) : undefined;
    },
    upsertPresence(p: Presence): Presence {
        upsertPresenceStmt.run(
            p.userId,
            p.isPresent ? 1 : 0,
            p.source,
            p.enteredAt,
            p.lastSeenAt,
        );
    }
};

```

```

    );
    return p;
  },
};

```

ステップ 6: IP 判定の改善

backend/src/middleware/ipCheck.ts (既存を確認):

```

import type { Request } from "express";
import dotenv from "dotenv";
dotenv.config();

export function getClientIp(req: Request): string {
  const fwd = req.headers["x-forwarded-for"];
  if (typeof fwd === "string" && fwd.length > 0) {
    return fwd.split(",")[0].trim();
  }
  return req.ip ?? "";
}

export function isLabIp(ip: string): boolean {
  const allowed = (process.env.LAB_ALLOWED_IPS ?? "")
    .split(",")
    .map((s) => s.trim())
    .filter(Boolean);
  if (allowed.length === 0) return false;
  return allowed.includes(ip);
}

```

Day1 で naganawa が実測した IP を .env の LAB_ALLOWED_IPS に書く。

ステップ 7: タイムアウト処理

backend/src/lib/timeout.ts (新規):

```

import { db } from "../db/database.js";

const TIMEOUT_MS = 10 * 60 * 1000; // 10分

// 10分 ping がないユーザーを自動退室
export function startTimeoutSweep() {
  const stmt = db.prepare(`

```

```

UPDATE presence
SET is_present = 0, entered_at = NULL
WHERE is_present = 1
    AND last_seen_at IS NOT NULL
    AND (julianday('now') - julianday(last_seen_at)) * 86400 * 1000 > ?
`);

```

```

setInterval(() => {
  const result = stmt.run(TIMEOUT_MS);
  if (result.changes > 0) {
    console.log(`[timeout] auto-checkout ${result.changes} user(s)`);
  }
}, 60 * 1000); // 1分ごとにチェック
}

```

backend/src/index.ts に追加:

```

import { startTimeoutSweep } from "../lib/timeout.js";
// app.listen() の後に
startTimeoutSweep();

```

ステップ 8: presence ルートの認証

naganawa が requireAuth ミドルウェアを作るので、それを使う。

backend/src/routes/presence.ts の userIdFromReq を req.user から取るように修正:

```

function userIdFromReq(req: import("express").Request): string | null {
  return req.user?.id ?? null;
}

```

そして backend/src/index.ts で:

```

import { requireAuth } from "../middleware/auth.js";
app.use("/api/presence", requireAuth, presenceRouter);

```

(naganawa が requireAuth を実装するのを待つ。先に進みたいなら手動で req.user をセットしておく)

ステップ 9: 動作確認

backend を起動

```
cd backend
```

```
npm run dev
```

別ターミナルで curl で API を叩く

```

curl http://localhost:3001/api/health
# → {"ok":true}

# (naganawa のログイン API ができてれば)
TOKEN=$(curl -s -X POST http://localhost:3001/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"name":"tsutsumi","password":"xxxx"}' | jq -r .token)

# ping
curl -X POST http://localhost:3001/api/presence/ping \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" -d '{}
# → {"userId":"u-tsutsumi","isPresent":true,"source":"wifi",...}

# 一覧
curl http://localhost:3001/api/presence
# → {"presences":[...]}
```

ステップ 10: PR を出す

```

git add backend/
git commit -m "feat: presence API を SQLite ベースで本実装、タイムアウト処理追加"
git push -u origin feature/tsutsumi_presence-db
```

第 5 部よくある詰まりどころ

better-sqlite3 のインストールでエラー

ネイティブビルドが必要。

```

xcode-select --install
rm -rf node_modules package-lock.json
npm install
```

SQLite ファイルが見つからない

カレントディレクトリの問題。npm run dev を必ず backend/ 直下で実行。

IP が常に ::1 になる

ローカル開発では正常。Mac は IPv6 のループバック。LAB_ALLOWED_IPS に ::1 を追加しておく。

CORS エラー

backend/src/index.ts で `app.use(cors());` が入っているか確認。

ping を打っても DB に書き込まれない

- ・ console.log で req.user が取れているか確認
- ・ requireAuth が機能しているか確認

第 6 部ここまでやったら他の人と合流

マイルストーン	誰と何を
ステップ 1 完了	naganawa: user テーブル設計を確定
ステップ 5 完了	naganawa: userStore から使えるように相談
ステップ 8 完了	takebayashi: API が動くこと、レスポンス形式が想定通りか確認
PR マージ後	全員: 動作確認

付録 A: チートシート

開発

```
cd backend && npm run dev
```

SQLite を直接見たい

```
sqlite3 backend/data/labsoldier.db
```

```
.tables
```

テーブル一覧

```
SELECT * FROM users;
```

```
SELECT * FROM presence;
```

```
.quit
```

DB をリセットしたいとき

```
rm backend/data/labsoldier.db
```

削除すれば次の起動で再作成

付録 B: 用語集

- ・ **SQL**: DB に問い合わせる言語
 - ・ **PRIMARY KEY**: テーブル内でユニークなキー
 - ・ **FOREIGN KEY**: 他テーブルの行を参照するキー
 - ・ **prepared statement**: SQL を事前にパース・最適化して使い回す仕組み (SQL インジェクション防止にも)
 - ・ **upsert**: insert or update (無ければ作る、あれば更新)
 - ・ **WAL**: SQLite の高速モード (Write-Ahead Logging)
-

この指南書を一通り終わると、在室判定の核ができます。困ったらすぐ **LINE** で相談。